

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Testes de Performance

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Planejamento e Execução de Testes de Performance

Testes de Performance
Aula 2





Olá!

Eu sou Luan Fonseca

Engenheiro de Qualidade de Software, atuando no Cesar há mais de 3 anos, e com 6 anos de experiência na área. Graduado em Sistemas de Informação pela UFRN, certificado pela ISTQB com a CTFL (Certified Tester Foundation Level), pós-graduado em Teste de Software, pela projeto Motorola/CIn-UFPE, pós graduando em Liderança Técnica pela CESAR School.

lof@cesar.school

<https://www.linkedin.com/in/luanof/>

O que veremos hoje?

- **Planejamento e Execução de Testes de Performance**
 - Principais atividades no teste de performance
 - Derivando os Objetivos do Teste de Performance
 - Processo:
 - Planejamento; Monitoramento e Controle dos Testes;
 - Análise e Modelagem do Teste;
 - Implementação; Execução e Conclusão dos Testes.
 - Prática com o Locust

Planejamento e Execução de Testes de Performance

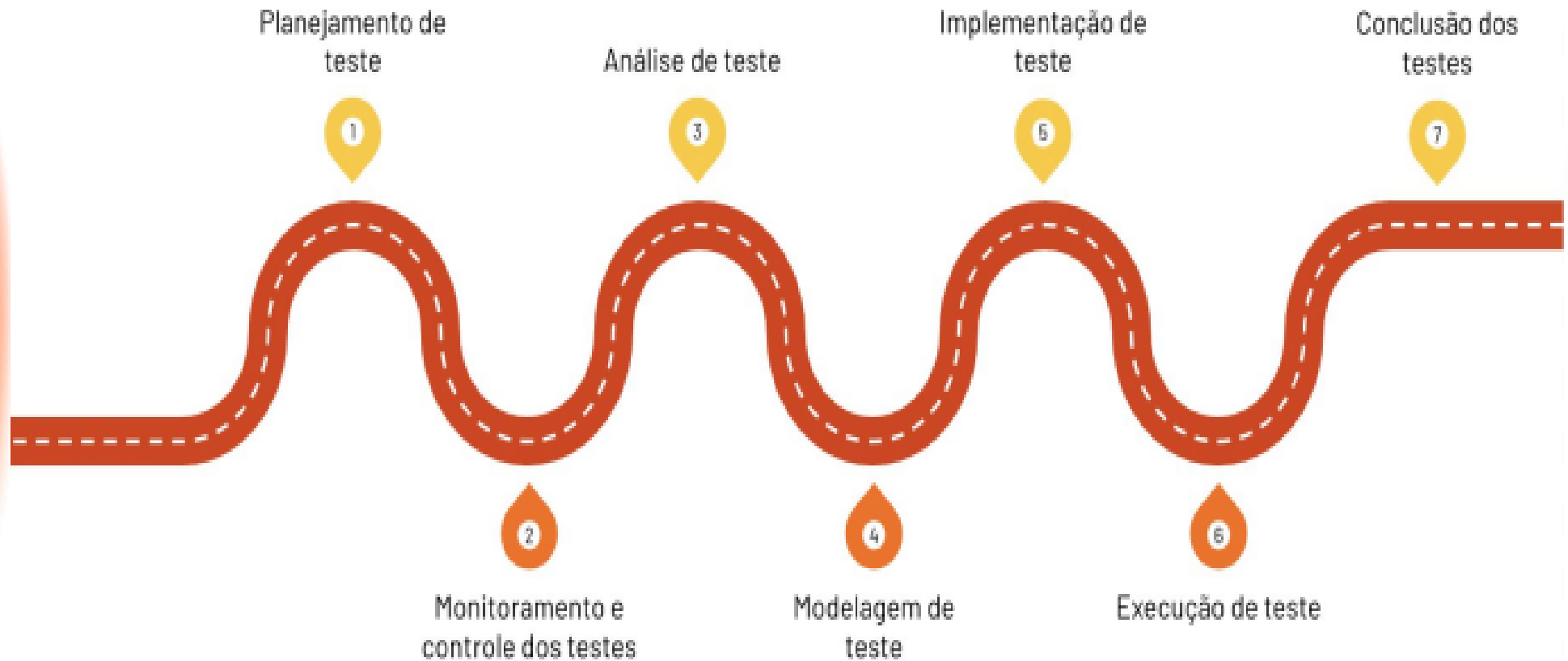
AVISO – FERIADOS!!

- **15 de novembro** (sábado – feriado da Proclamação da República) **Não haverá aula**
- **21 e 22 de novembro** (sexta e sábado – dia 20/11 Dia da Consciência Negra impressado) **Não haverá aula**
- As turmas das sextas e sábado esta semana terão aula apenas na sexta dia 14/11 e depois retomam as aulas em **28/11**,

Principais atividades no teste de performance

- Teste de performance é iterativo.
- Cada teste fornece dados valiosos sobre a performance do sistema.
- Informações coletadas são usadas para corrigir ou otimizar o aplicativo e o sistema.
- Próxima iteração de teste mostra os resultados das modificações.
- Iterações contínuas até que os objetivos do teste sejam atingidos.

Processo



Processo

Planejamento de Teste:

- Essencial para alocação de ambientes, dados, ferramentas e recursos humanos.
- Estabelece o escopo do teste de performance.
- Conclui a identificação e análise de risco, atualizando a documentação de planejamento de teste.
- Revisão e modificação contínua do planejamento conforme as condições de risco mudam.

Processo

Monitoramento e Controle dos Testes:

As medidas de controle são definidas para fornecer planos de ação, caso sejam encontrados problemas que possam afetar a eficiência da performance, como:

- O aumento da capacidade de geração de carga se a infraestrutura não gerar as cargas desejadas, conforme planejado para testes de performance específicos.
- O hardware modificado, novo ou substituído.
- As alterações nos componentes de rede.
- As mudanças na implementação de software.

Avaliação dos objetivos de teste de performance para verificar critérios de saída

Processo

Análise de Teste:

- Baseada em requisitos de performance, SLA, arquitetura de TI, modelos de processo.
- Identificação de condições específicas de teste (carga, tempo, transações).
- Decisão sobre tipos de teste de performance necessários (carga, estresse, escalabilidade).

Modelagem de Teste:

- Projeto de casos de teste de performance modulares.
- Implementação ordenada dos casos de teste em procedimentos.
- Estabelecimento de ambiente e dados de teste representativos da produção.

Processo

Execução de Teste:

- Realização dos testes de performance usando ferramentas especializadas.
- Avaliação dos resultados para verificar cumprimento dos requisitos.
- Relato de quaisquer defeitos encontrados.

Conclusão dos Testes:

- Relatório de resumo de teste entregue aos stakeholders.
- Resultados expressos através de métricas e dashboards para melhor entendimento.
- Arquivamento dos testwares para uso em atividades de manutenção do sistema.



PLANEJAMENTO



Derivando os Objetivos do Teste de Performance

Stakeholders:

- Incluem usuários e pessoas com experiência comercial ou técnica.
- Têm objetivos diferentes relacionados ao teste de performance.
- Definem objetivos, terminologia e critérios para determinar se os objetivos foram alcançados.

Objetivos de Usuários:

- Concentram-se na satisfação do usuário final e nos objetivos de negócio.
- Foco em permitir que os usuários façam o que precisam fazer sem preocupações com os recursos ou a entrega do produto.

Derivando os Objetivos do Teste de Performance

Objetivos Técnicos:

- Concentram-se nos aspectos operacionais do sistema.
- Respondem a perguntas sobre capacidade de escalonamento e condições de degradação de performance.

Principais Objetivos do Teste de Performance:

- Identificação de riscos potenciais.
- Descoberta de oportunidades de melhoria.
- Identificação das mudanças necessárias.

Derivando os Objetivos do Teste de Performance

Ao coletar informações dos vários stakeholders, as seguintes perguntas devem ser respondidas:

- Quais transações serão executadas no teste de performance e qual tempo médio de resposta é esperado?
- Quais métricas do sistema devem ser capturadas (p. ex., uso de memória, taxa de transferência da rede) e quais valores são esperados?
- Quais melhorias de performance são esperadas desses testes em comparação com ciclos de testes anteriores?



Bombardeie sua API com Locust 🚀

Locust permite que você teste a sua aplicação simulando um grande número de usuários, como um verdadeiro enxame de gafanhotos. Utilize a interface web intuitiva ou o modo de linha de comando para monitorar a performance e identificar gargalos na sua aplicação.



Locust

Ferramenta de Teste de Carga

- Locust é uma ferramenta de código aberto para realizar testes de carga distribuídos.
- Permite simular milhões de usuários simultâneos.
- Código Simples: Crie seus testes diretamente em código Python. E adicione recursos extras com o poder do Python.
- Distribuído e Escalável: Suporta teste distribuído, permitindo escalabilidade de usuários conectados.

Locust

Potencialidades da Ferramenta

Principais Características:

- **Distribuição:** Suporta testes distribuídos para simular um grande número de usuários.
- **Escalabilidade:** Escala facilmente conforme a necessidade.
- **Flexibilidade:** Scripts de teste em Python permitem alta personalização.
- **Interface Web:** Oferece uma interface web amigável para monitoramento em tempo real.
- **Relatórios Detalhados:** Gera relatórios abrangentes para análise de resultados.

O que são Testes de Carga?

(Relembrando)

Definição

- Testes de carga avaliam a performance de um sistema sob condições de uso realistas.
- Identificam como o sistema lida com altos volumes de tráfego.

Objetivos

- Determinar a capacidade máxima de operação.
- Identificar gargalos de performance.
- Garantir que o sistema atende aos requisitos de performance.

Exemplo de código

```
locustfile.py

from locust import HttpUser, between, task

class WebsiteUser(HttpUser):
    wait_time = between(5, 15)

    def on_start(self):
        self.client.post("/login", {
            "username": "test_user",
            "password": ""
        })

    @task
    def index(self):
        self.client.get("/")
        self.client.get("/static/assets.js")

    @task
    def about(self):
        self.client.get("/about/")
```

```
$ locust -f locustfile.py
```

Exemplo de código

```
locustfile.py

from locust import HttpUser, between, task

class WebsiteUser(HttpUser):
    wait_time = between(5, 15)

    def on_start(self):
        self.client.post("/login", {
            "username": "test_user",
            "password": ""
        })

    @task
    def index(self):
        self.client.get("/")
        self.client.get("/static/assets.js")

    @task
    def about(self):
        self.client.get("/about/")
```

```
$ locust -f locustfile.py
```

Principais Elementos:

1. **Importações:**
 - a. `HttpUser`, `between`, `task`: Importados da biblioteca Locust.
2. **Classe `WebsiteUser`:**
 - a. **Herança:** Deriva de `HttpUser` para simular um usuário HTTP.
 - b. **`wait_time`:** Define o intervalo de espera entre as tarefas, entre 5 e 15 segundos.
3. **Método `on_start`:**
 - a. **Autenticação:** Executa uma requisição POST para `/login` com as credenciais do usuário no início de cada simulação.
4. **Tarefa `index`:**
 - a. **Requisições GET:** Simula a visita à página principal (`/`) e a solicitação de um arquivo JavaScript (`/static/assets.js`).
5. **Tarefa `about`:**
 - a. **Requisição GET:** Simula a visita à página "Sobre" (`/about/`).

Web Mode



HOST
https://fakerestoreapi.azurewebsites.net

STATUS
RUNNING
10 users
[Edit](#)

RPS
1.1

FAILURES
0%



[Statistics](#) [Charts](#) [Failures](#) [Exceptions](#) [Download Data](#)

Type	Name	# requests	# fails	Median (ms)	Average (ms)	Min (ms)	Max (ms)	Content Size (bytes)	# reqs/sec
GET	/api/authors	14	0	230	239	225.0375747680664	331.0878276824951	49623	0.5
GET	/api/books	17	0	250	456	227.29039192199707	1006.1233043670654	97253	0.4
GET	/api/users	15	0	240	527	219.88582611083984	1316.4410591125488	524	0.2
Total		46	0	240	413	219.88582611083984	1316.4410591125488	51215	1.1

Primeiros Passos com Locust

Sites e APIs de Exemplos

- <http://fakerestapi.azurewebsites.net/index.html>
- <https://demoblaze.com/index.html>
- <https://blazedemo.com/index.php>

INTERVALO!



15 min.



Instalar Python3

MacOS

```
bash

# Instalar Homebrew (se não tiver)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Instalar Python 3
brew install python

# Verificar versão
python3 --version
```

Windows

```
cmd

# Passos:
# 1. Baixar instalador em: https://www.python.org/downloads/windows/
# 2. Marcar "Add Python to PATH" na instalação
# 3. Testar no Prompt de Comando (cmd):

python --version

# ou

python3 --version
```

Linux

Ubuntu/Debian

```
bash

sudo apt update
sudo apt install python3 python3-pip -y
python3 --version
```

Fedora

```
bash

sudo dnf install python3 python3-pip -y
python3 --version
```

Arch

```
bash

sudo pacman -S python python-pip
python3 --version
```

Primeiros Passos com Locust

Instalação

- Instalar o Visual Studio Code (VS Code)
- Utilizando o terminal (pode ser de uma IDE)
- Instale Python (versão 3.7 ou superior).
- Crie um diretório para o projeto (ex: performanceLocust)
- Crie um Virtual environment
 - `python3 -m venv venv | ou | python -m venv venv | ou | py -m venv venv`
- Ative o Virtual Environment
 - `source venv/bin/activate | ou | venv\Scripts\activate`
- Use pip para instalar Locust
 - `pip3 install locust | ou | pip install locust`

Primeiros Passos com Locust

Criando um Teste Simples:

- Criar um arquivo `.py` (ex: `performanceTests.py`)
- Importe Locust:
 - `from locust import HttpUser, task, between`

- 

```
python Copy code  
  
class WebsiteUser(HttpUser):  
    wait_time = between(1, 6)  
  
    @task  
    def index(self):  
        self.client.get("/")
```

- Execute o comando no terminal:
 - `locust -f nome_do_arquivo.py`
- Acesse a Interface Web:
 - Abra o navegador e vá para <http://localhost:8089>.
- `host = "https://fakerestapi.azurewebsites.net/api/v1"`

Primeiros Passos com Locust

Executando um Teste Locust pela Linha de Comando

- Comando Básico
 - `locust -f nome_do_seu_arquivo.py`
- Principais Opções e Argumentos:
 - Especificar o arquivo do teste:
 - `-f, --locustfile <arquivo.py>`
 - Definir número de usuários simulados:
 - `-u, --users <número>`
 - Taxa de incremento de usuários por segundo:
 - `-r, --spawn-rate <número>`
 - Host da aplicação a ser testada:
 - `--host <url>`
 - Executar teste em modo sem interface web:
 - `--headless`
 - Duração do teste:
 - `--run-time <tempo> (ex: 1h30m)`
 - Salvar resultados em um arquivo CSV:
 - `--csv <nome_do_arquivo>`

Primeiros Passos com Locust

- **Exemplo Completo::**

- `locust -f teste_locust.py -u 100 -r 10 --host http://minhaaplicacao.com --headless --run-time 60s --csv resultados`

- **Descrição:**

- O comando acima executa um teste definido no arquivo teste_locust.py com 100 usuários, aumentando 10 usuários por segundo, na URL especificada, por 1 hora, e salva os resultados em arquivos CSV.

Locust

Contato e Recursos:

- Documentação oficial: locust.io
- Comunidade e suporte: GitHub, Stack Overflow, Fóruns Locust



**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados